

MERN STACK OVERVIEW

1. What is MERN ?

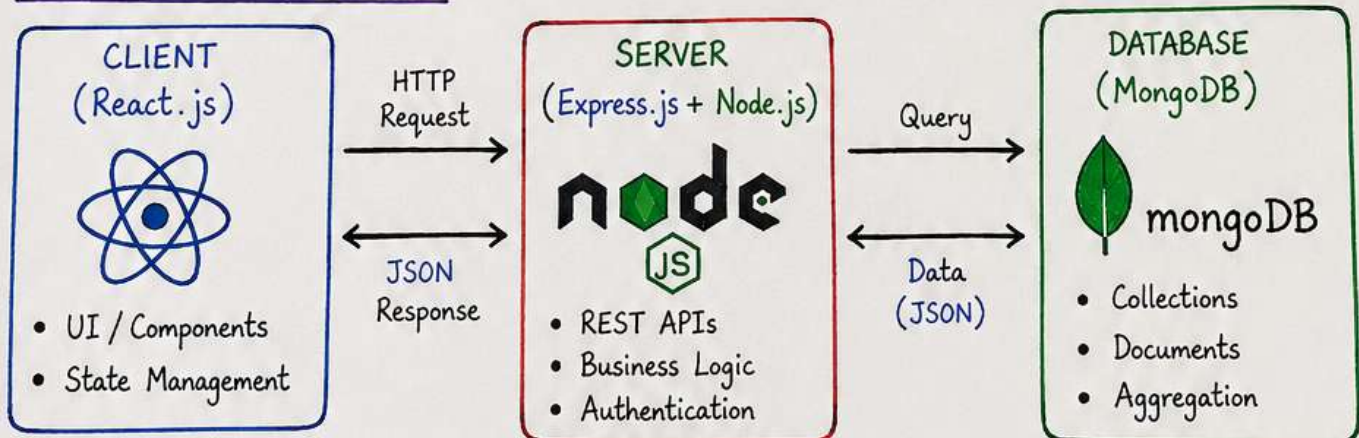
MERN is a JavaScript stack used to build modern, full-stack web applications. It consists of four technologies that work together to build dynamic and efficient web apps.

- M** - MongoDB
(Database)
- E** - Express.js
(Backend Framework)
- R** - React.js
(Frontend Library)
- N** - Node.js
(Runtime Environment)

2. Why MERN is Popular ?

- Uses JavaScript everywhere (Frontend + Backend)
- High performance and scalability
- Flexible and easy to learn
- Large ecosystem and active community
- Perfect for Single Page Applications (SPA)
- Open source and completely free

3. MERN Architecture



4. Request Flow in MERN

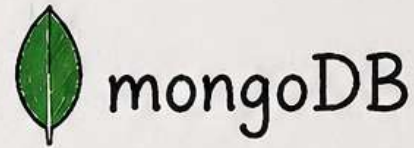
- ① User interacts with React UI.
- ② React sends HTTP request to Express API.
- ③ Express (running on Node.js) processes the request.
- ④ Express interacts with MongoDB to fetch/store data.
- ⑤ Data is sent back to Express.
- ⑥ Express sends JSON response back to React.
- ⑦ React updates UI with the received data.

Key Benefits of MERN

- ★ End-to-end JavaScript
- ★ Faster Development
- ★ Reusable Code
- ★ Better Performance
- ★ Ideal for SPAs

MONGODB BASICS

1. What is MongoDB?



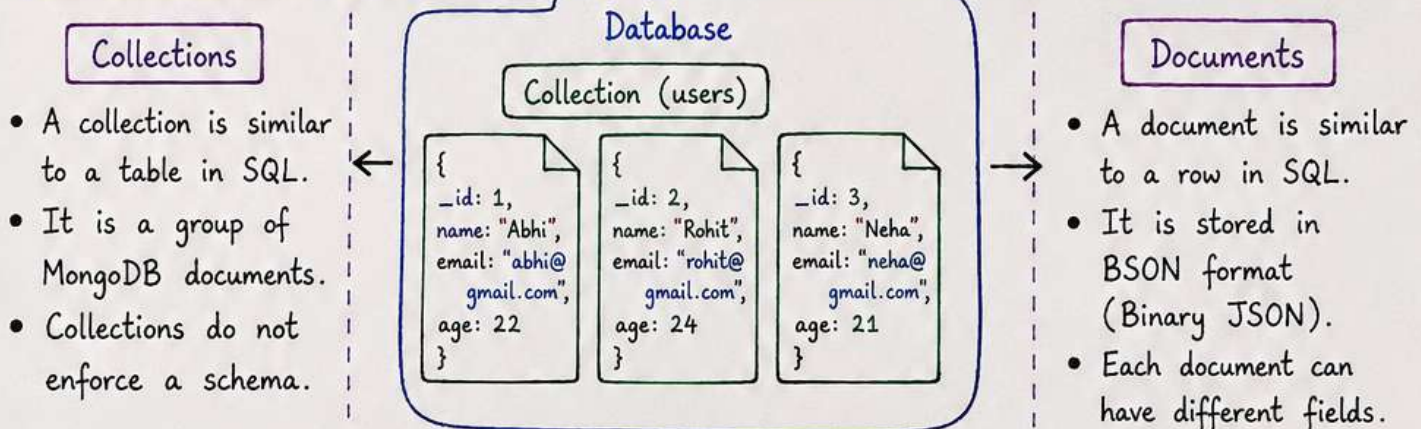
MongoDB is a NoSQL, document-oriented database. Instead of storing data in tables (like SQL), it stores data in flexible, JSON-like documents.

- ✓ Schema-less (Flexible Schema)
- ✓ High Performance
- ✓ Scalable & Highly Available
- ✓ Open Source
- ✓ Cross Platform

2. NoSQL Database

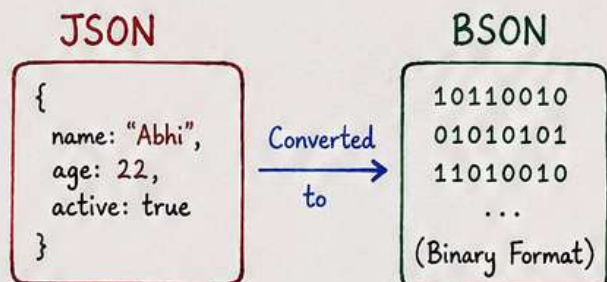
- NoSQL stands for "Not Only SQL".
- It is used for unstructured or semi-structured data.
- It is designed for large scale, high speed applications.
- MongoDB is the most popular NoSQL database.

3. Collections vs Documents



4. BSON (Binary JSON)

- MongoDB stores data in BSON format.
- BSON is a binary representation of JSON.
- It supports more data types than JSON.
- It is faster to read/write compared to JSON.



Why BSON?

- ★ Supports more data types (Date, ObjectId, Binary, etc.)
- ★ Faster processing
- ★ Efficient storage
- ★ Better for large & complex data

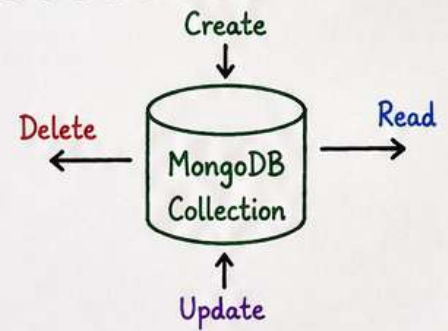


Key Takeaway

MongoDB stores data in collections and documents using BSON format, which makes it flexible, scalable and high performance.

MONGODB CRUD OPERATIONS

CRUD stands for Create, Read, Update and Delete. MongoDB provides simple methods to perform these operations on collections.



① INSERT ONE()

- Inserts a single document
- Returns the inserted document id.

```
// Insert a single document
db.users.insertOne({
  name: "Abhi",
  email: "abhi@gmail.com",
  age: 22,
  city: "Pune"
})
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId("662a...")
}
```

② FIND()

- Retrieves documents
- Without filter → all documents
- With filter → matching docs

```
// Find all documents
db.users.find()

// Find with filter
db.users.find({ age: { $gt: 20 } })
```

Example Output:

```
[
  { _id: ObjectId("1"), name: "Abhi", age: 22 },
  { _id: ObjectId("2"), name: "Rohit", age: 24 }
]
```

③ UPDATE ONE()

- Updates the first document that matches the filter.
- Use \$set to update fields.

```
// Update a document
db.users.updateOne(
  { name: "Abhi" },
  { $set: { age: 23, city: "Mumbai" } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

④ DELETE ONE()

- Deletes the first document that matches the filter.

```
// Delete a document
db.users.deleteOne({ name: "Abhi" })
```

Output:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

COMMON EXAMPLES

Operation	Method	Example	Description
Insert	insertOne()	db.users.insertOne({ name: "A" })	Insert a single document
Read (all)	find()	db.users.find()	Get all documents
Read (filter)	find(filter)	db.users.find({ age: { \$gt: 20 } })	Get matching documents
Update	updateOne()	db.users.updateOne({ name: "A" }, { \$set: { age: 23 } })	Update first matched document
Delete	deleteOne()	db.users.deleteOne({ name: "A" })	Delete first matched document



NOTE:

By default, MongoDB creates an _id field (ObjectId) for each document which is unique.

IMPORTANT POINTS

- ✓ Collection is created automatically when first document is inserted.
- ✓ Field names are case-sensitive.
- ✓ Documents in a collection can have different fields.

MONGODB CRUD OPERATIONS

1. CRUD in MongoDB

MongoDB provides 4 main operations:

- Create (Insert)
- Read (Find)
- Update (Modify)
- Delete (Remove)

CRUD Operations Summary

- C** → Create : insertOne() / insertMany()
- R** → Read : find() / findOne()
- U** → Update : updateOne() / updateMany()
- D** → Delete : deleteOne() / deleteMany()

2. Create (Insert Documents)

- insertOne() - Inserts a single document.

```
db.users.insertOne({
  name: "Abhi",
  age: 22,
  skills: ["JavaScript", "React"]
})
```

- insertMany() - Inserts multiple documents.

```
db.users.insertMany([
  { name: "Abhi", age: 22 },
  { name: "Rohit", age: 23 }
])
```

3. Read (Find Documents)

- find() - Returns all matching documents.

```
db.users.find({ age: 22 })
```

- findOne() - Returns only first match.

```
db.users.findOne({ name: "Abhi" })
```

Query Examples:

- db.users.find() // All documents
- db.users.find({ age: 22 }) // Age = 22
- db.users.find({ age: { \$gt: 20 } }) // age > 20
- db.users.find({ name: /Abhi/ }) // Name contains Abhi

4. Update (Modify Documents)

- updateOne() - Updates first matching document.

```
db.users.updateOne(
  { name: "Abhi" },
  { $set: { age: 23 } }
)
```

- updateMany() - Updates all matching documents.

```
db.users.updateMany(
  { department: "IT" },
  { $set: { status: "Active" } }
)
```

5. Delete (Remove Documents)

- deleteOne() - Deletes first matching document.

```
db.users.deleteOne({ name: "Abhi" })
```

- deleteMany() - Deletes all matching documents.

```
db.users.deleteMany({ status: "Inactive" })
```

6. Useful Tips

- ✓ MongoDB stores data in JSON-like format (BSON).
- ✓ insertMany() is more efficient than multiple insertOne().
- ✓ Use indexes for better query performance.
- ✓ Always validate data before storing.



mongoDB

MONGODB ADVANCED

1. INDEXING

- Indexing improves the speed of data retrieval operations on a collection.
- MongoDB uses indexes similar to a book index.
- Types of Indexes:
 - Single Field Index
 - Compound Index
 - Unique Index
 - Text Index
 - TTL Index (Time To Live)
 - Geospatial Index

```
// Create index on name field
db.users.createIndex({ name: 1 })

// Create compound index
db.users.createIndex({
  age: 1,
  city: 1
})
```

Benefits

- ★ Faster queries
- ★ Efficient sorting
- ★ Improved performance
- ★ Supports unique constraints
- ★ Reduces load on server

Note:

Too many indexes can slow down writes and consume more space.



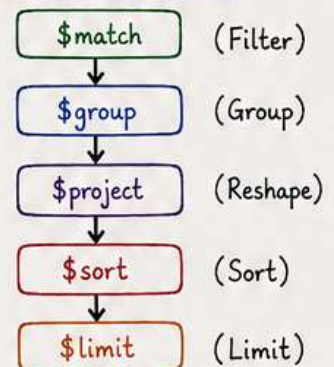
2. AGGREGATION

- Aggregation is used to process data and return computed results.
- It works on a pipeline of stages.
- Common Stages:
 - \$match - Filter documents
 - \$group - Group documents
 - \$sort - Sort documents
 - \$project - Reshape documents
 - \$limit - Limit results
 - \$skip - Skip documents

Example: Total users in each city

```
db.users.aggregate([
  { $group: {
    _id: "$city",
    totalUsers: { $sum: 1 }
  } },
  { $sort: { totalUsers: -1 } }
])
```

Aggregation Pipeline



3. SCHEMA DESIGN

- Good schema design improves performance and makes data management easy.
- Keep it simple and follow application needs.
- Consider read/write patterns.
- Avoid deep nesting.

Tips

- ★ Identify core entities
- ★ List required fields
- ★ Decide relationships
- ★ Choose embedding or referencing
- ★ Index frequently queried fields

4. RELATIONSHIPS

MongoDB is NoSQL, so it does not have JOINS like SQL. It uses two approaches to handle relationships.

When to use which?

- Embedding → One-to-few, data changes together.
- Referencing → One-to-many, large or frequently changing data.

A) EMBEDDING

- Related data is stored inside the same document.
- Good for one-to-few relationships.
- Faster reads.
- Example:

```
{
  _id: 1,
  name: "Abhi",
  email: "abhi@gmail.com",
  addresses: [
    { city: "Pune", pin: 411001 },
    { city: "Mumbai", pin: 400001 }
  ]
}
```

B) REFERENCING

- Related data is stored in a separate collection.
- Good for one-to-many relationships.
- Avoids data duplication.
- Example:

Users Collection

```
{
  _id: 1,
  name: "Abhi",
  email: "abhi@gmail.com",
  postIds: [101, 102]
}
```

Posts Collection

```
{ _id: 101, userId: 1, title: "Post 1" }
{ _id: 102, userId: 1, title: "Post 2" }
```

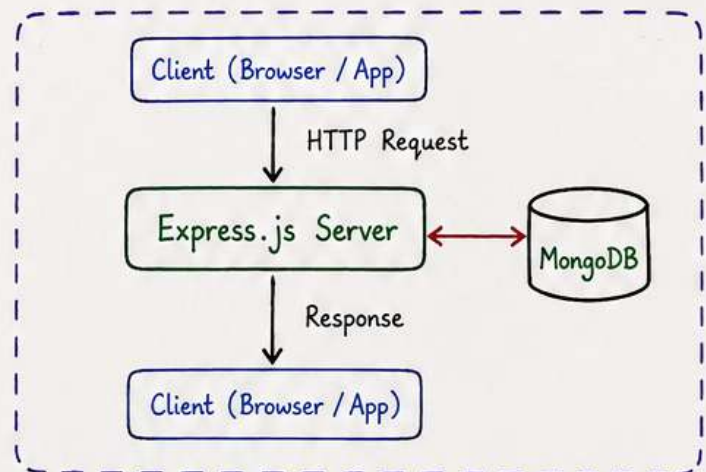
★ Key Takeaway: Use Indexing for performance, Aggregation for complex queries, Design schema carefully, and choose Embedding or Referencing based on your use case.



EXPRESS.JS INTRODUCTION

1. What is Express.js ?

- Express.js is a fast, unopinionated, minimal and flexible Node.js web application framework.
- It provides a thin layer of fundamental web application features for Node.js.
- It helps us build APIs and web applications quickly and easily.



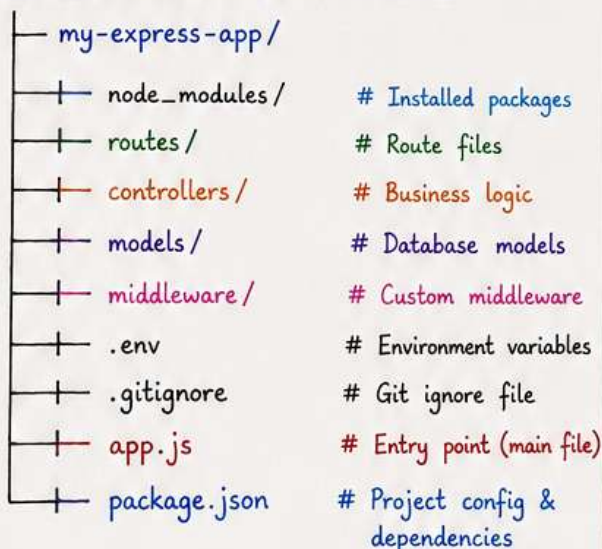
2. Key Features

- ★ Fast and lightweight
- ★ Minimal and flexible
- ★ Easy to set up routing
- ★ Middleware support
- ★ Handles HTTP requests and responses
- ★ Works well with templates and databases
- ★ Huge community support

3. Why Express.js ?

- ✓ Makes server-side development easier with Node.js
- ✓ Simplifies routing and middleware handling
- ✓ Provides useful utilities and helpers
- ✓ Highly extensible
- ✓ Perfect for building REST APIs and backends

4. Folder Structure (Basic)



5. First Express Server

Step 1: Initialize project

```
npm init -y
```

Step 2: Install express

```
npm install express
```

Step 3: Create app.js

```
const express = require('express');
const app = express();
const PORT = 5000;

app.get('/', (req, res) => {
  res.send('Hello, Express.js! 🚀');
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

6. Run the Server

```
node app.js
```

Output in terminal:

```
Server is running on port 5000
```



Note:

Open your browser and go to

```
http://localhost:5000
```

You will see: Hello, Express.js! 🚀

Tip: Express is unopinionated → It gives you the freedom to structure your project as you like. ★

EXPRESS.JS ROUTING

Routing in Express is used to define the endpoints (URLs) of your application and the HTTP methods that can be used to access them.

Basic Syntax

app.METHOD(PATH, HANDLER)



1. GET - Retrieve Data

- Used to get data from the server.
- Should not change any data.

```
// Example : Get all users
app.get('/users', (req, res) => {
  res.send("List of all users");
});
```

Example Request

GET http://localhost:5000/users

Example Response

List of all users

2. POST - Create Data

- Used to send data to the server to create a new resource.
- Data is usually sent in the request body.

```
// Example : Create a new user
app.post('/users', (req, res) => {
  const newUser = req.body;
  res.status(201).send("User created");
});
```

Example Request

POST http://localhost:5000/users
Body: { "name": "Abhi", "email": "abhi@gmail.com" }

Example Response

User created

3. PUT - Update Data (Full Update)

- Used to update an existing resource completely.
- The entire resource should be sent in body.

```
// Example : Update user by id
app.put('/users/1', (req, res) => {
  res.send("User updated completely");
});
```

Example Request

PUT http://localhost:5000/users/1
Body: { "name": "Abhi Kumar", "email": "abhi@x.com" }

Example Response

User updated completely

4. PATCH - Update Data (Partial Update)

- Used to update a resource partially.
- Only the fields to be updated are sent.

```
// Example : Partially update user
app.patch('/users/1', (req, res) => {
  res.send("User updated partially");
});
```

Example Request

PATCH http://localhost:5000/users/1
Body: { "email": "newemail@x.com" }

Example Response

User updated partially

5. DELETE - Delete Data

- Used to delete a resource from the server.

```
// Example : Delete user by id
app.delete('/users/1', (req, res) => {
  res.send("User deleted");
});
```

Example Request

DELETE http://localhost:5000/users/1

Example Response

User deleted



Tip:

Always use proper HTTP methods. It makes your API RESTful, clean and easy to maintain.

6. Route Parameters

- Used to capture values in the URL.
- Use : (colon) before the parameter name.

```
// Example : Get user by id
app.get('/users/:id', (req, res) => {
  const id = req.params.id;
  res.send(`User id is ${id}`);
});
```

Example Request

GET http://localhost:5000/users/25

Example Response

User id is 25

7. Query Parameters

- Used to send optional data in the URL.
- Access using req.query.

```
// Example : Search users
app.get('/users', (req, res) => {
  const name = req.query.name;
  res.send(`Search result for ${name}`);
});
```

Example Request

GET http://localhost:5000/users?name=abhi

Example Response

Search result for abhi

? starts the query string. Multiple params are separated by &

★ Quick Summary

GET → Read
POST → Create

PUT → Update (Full)
PATCH → Update (Partial)

DELETE → Delete



EXPRESS.JS MIDDLEWARE

Middleware functions are functions that have access to the request object (**req**), the response object (**res**), and the **next** middleware function in the application's request-response cycle.

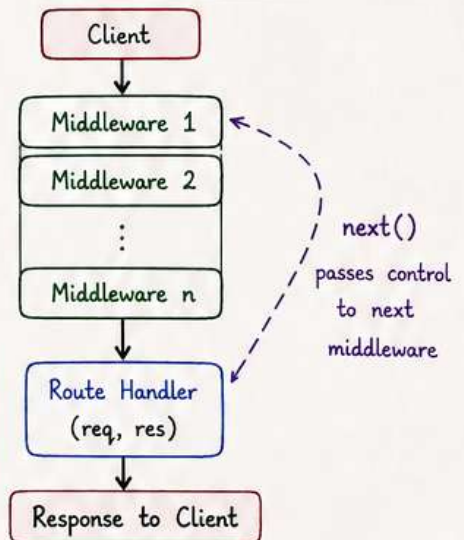
1. BUILT-IN MIDDLEWARE

- Express comes with lots of built-in middleware.
- Examples: `express.json()`, `express.urlencoded()`, `express.static()`, `express.text()`, etc.
- These handle common tasks like parsing request bodies, serving static files, etc.

Example : Using `express.json()`

```
const express = require('express');
const app = express();
app.use(express.json()); // to parse JSON data
app.post('/api/data', (req, res) => {
  res.send(req.body);
});
```

Middleware in Action



2. CUSTOM MIDDLEWARE

- We can create our own middleware functions.
- They can perform any task (logging, authentication, validation, etc.) before passing control.

Example : Custom Logger Middleware

```
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // pass control to next middleware
};
app.use(logger);
```

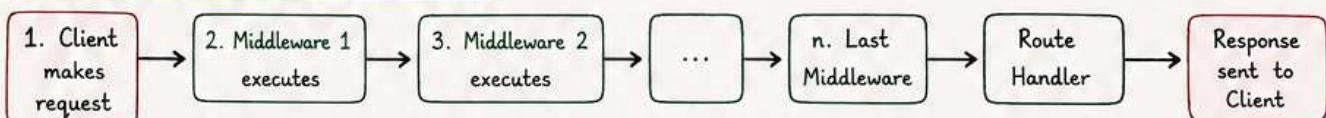
3. ERROR HANDLING MIDDLEWARE

- Used to handle errors in the application.
- It has four parameters: (**err**, **req**, **res**, **next**)
- Must be defined after all other middleware and routes.

Example : Error Handling Middleware

```
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send({ message: "Something went wrong!" });
};
app.use(errorHandler);
```

4. REQUEST LIFECYCLE IN EXPRESS



If any middleware does not call `next()` or send a response, the request will hang!

5. TYPES OF MIDDLEWARE

- Application-level middleware
→ Bound to app instance using `app.use()`
- Router-level middleware
→ Bound to Express Router using `router.use()`
- Error-handling middleware
→ Handles errors in the application
- Built-in middleware
→ Provided by Express
- Third-party middleware
→ Installed via npm

6. MIDDLEWARE STACK EXAMPLE

```
const express = require('express');
const app = express();

app.use(express.json()); // Built-in
app.use(logger);          // Custom
app.use(authMiddleware);  // Custom
                           } Middleware Stack

app.get('/home', (req, res) => { // Route
  res.send("Welcome Home!");
});                           } Route Handler

app.use(errorHandler);       // Error handler
                           } Error Handling Middleware
```

★ **NOTE** : Order matters in middleware.
The request flows from top to bottom.
Each middleware must call `next()` to pass control.



Tip : Use middleware for Cross-Cutting Concerns like logging, authentication, validation, etc.
It keeps your code clean and reusable.

REST APIs

1. What is REST ?

- **REST** (Representational State Transfer) is an architectural style for designing networked applications.
- It uses **HTTP** methods to perform CRUD operations on resources.
- It is **Stateless**, scalable and follows standard conventions.

2. REST Principles

- ① **Client-Server Architecture**
Separation of concerns between client (UI) and server (data).
- ② **Stateless**
Each request from client must contain all the information needed.
- ③ **Cacheable**
Responses should be cacheable to improve performance.
- ④ **Uniform Interface**
Consistent way of communication using standard methods.
- ⑤ **Layered System**
Client cannot tell if it is connected directly to end server or an intermediary.
- ⑥ **Code on Demand (Optional)**
Server can extend functionality on client (e.g., JS).

3. HTTP Methods in REST

Method	Purpose	Idempotent	Safe
GET	Retrieve a resource	Yes	Yes
POST	Create a resource	No	No
PUT	Update (Replace) a resource	Yes	No
PATCH	Update (Partial) a resource	Yes	No
DELETE	Delete a resource	Yes	No

Idempotent → Multiple requests have the same effect (e.g., GET, PUT, DELETE)
Safe → Does not modify data (e.g., GET)

4. RESTful URL Design Best Practices

- ✓ Use nouns, not verbs.
- ✓ Use plural nouns.
- ✓ Use lowercase letters.
- ✓ Use hyphens for better readability.
- ✓ Use path parameters for specific resources.
- ✓ Use query parameters for filtering, sorting, pagination.

Examples :

GET /api/users → Get all users
 GET /api/users/25 → Get user by id
 POST /api/users → Create a new user
 PUT /api/users/25 → Update user (full)
 PATCH /api/users/25 → Update user (partial)
 DELETE /api/users/25 → Delete user
 GET /api/users?role=admin&sort=name
 → Get users with role=admin sorted by name

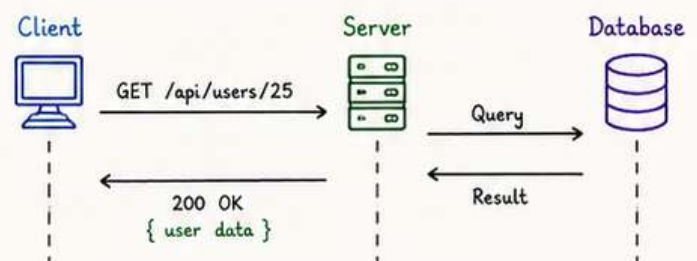
5. Status Codes

Code Range	Status	Meaning	Examples
1xx	Informational	Request received, processing	100 Continue
2xx	Success	Request successful	200 OK, 201 Created
3xx	Redirection	Further action needed	301 Moved Permanently, 304 Not Modified
4xx	Client Error	Invalid request from client	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Error	Server failed to fulfill request	500 Internal Server Error, 503 Service Unavailable

6. CRUD Mapping in REST

Operation	HTTP Method	Endpoint (Example)	Description
Create	POST	/api/users	Create a new user
Read (All)	GET	/api/users	Get all users
Read (One)	GET	/api/users/25	Get user by id
Update (Full)	PUT	/api/users/25	Update entire user
Update (Partial)	PATCH	/api/users/25	Update partial user
Delete	DELETE	/api/users/25	Delete user

7. Example REST API Flow (Get User)



Key Takeaway : REST APIs provide a standard way for apps to communicate. Follow the principles, use proper methods, status codes and URL design for clean, scalable and maintainable APIs.



EXPRESS.JS REQUEST & RESPONSE

In Express.js, every request coming to the server is handled by a **Route Handler**, which receives a **Request (req)**, and sends back a **Response (res)**.

1. REQUEST (req) OBJECT

The req object contains information about the incoming request.

Common Properties

- **req.method** → HTTP method (GET, POST, etc.)
- **req.url** → URL of the request
- **req.headers** → Headers sent by the client
- **req.params** → Route parameters
- **req.query** → Query string parameters
- **req.body** → Data sent in request body
- **req.ip** → IP address of the client
- **req.cookies** → Cookies sent by the client
- **req.files** → Uploaded files (with middleware)

Example : Accessing Request Data

```
app.get('/user/:id', (req, res) => {
  // Route parameter
  console.log(req.params.id);
  // Query parameter
  console.log(req.query.name);
  // Headers
  console.log(req.headers['user-agent']);
  // Request body (for POST/PUT)
  console.log(req.body);
});
```

Example Request

```
GET /user/25?name=abhi
Host: localhost:5000
User-Agent: Chrome
...
```



Available in req

```
req.params.id = 25
req.query.name = "abhi"
req.headers["user-agent"]
= "Chrome"
```

2. RESPONSE (res) OBJECT

The res object is used to send a response back to the client.

Common Methods

- **res.status(code)** → Set HTTP status code
- **res.send(data)** → Send text, JSON or HTML
- **res.json(obj)** → Send JSON response
- **res.sendFile(path)** → Send a file
- **res.redirect(url)** → Redirect to another URL
- **res.end()** → End the response

Example : Sending Response

```
app.get('/success', (req, res) => {
  res.status(200).json({
    message: "Request successful",
    data: [1, 2, 3]
  });
});
// Send plain text
res.send("Hello, Abhi!");
// Redirect
res.redirect("/login");
```

Common Status Codes

200	OK - Success
201	Created - Resource created
400	Bad Request
401	Unauthorized
404	Not Found
500	Internal Server Error



Tip

Always send a meaningful status code with your response.

3. REQUEST BODY PARSING

Express does not parse request body by default. Use middleware to parse it.

```
// For JSON data
app.use(express.json());

// For URL-encoded data
app.use(express.urlencoded({ extended: true }));
```

Without Body Parser Middleware ❌

req.body will be undefined.

With Body Parser Middleware ✅

req.body will contain the data.

Example POST Request (JSON)

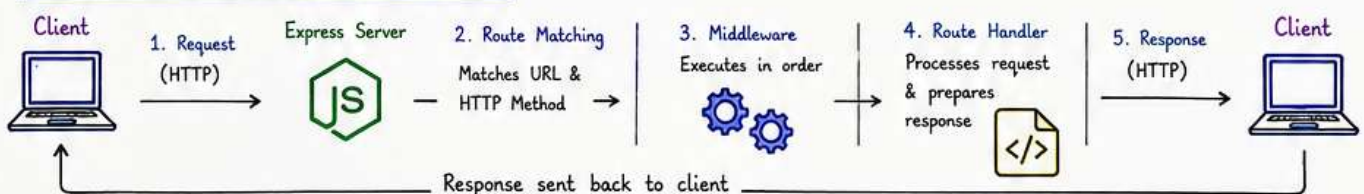
```
POST /api/users
Content-Type: application/json
{
  "name": "Abhi",
  "email": "abhi@gmail.com"
}
```



Available in req.body

```
{ name: "Abhi", email: "abhi@gmail.com" }
```

4. FLOW OF REQUEST & RESPONSE



Key Takeaway :

- req object → contains related data.
- res object → used to send response to the client.

- Use body parser middleware to read req.body.
- Always send appropriate status codes.



EXPRESS.JS ROUTERS

In Express.js, a **Router** is a modular, mountable route handler. It helps us organize routes in separate files for better structure and maintainability.

RESTful Mapping

GET	→ Read
POST	→ Create
PUT	→ Update (Full)
PATCH	→ Update (Partial)
DELETE	→ Delete

1. Why Use Routers?

- Keeps code organized and modular.
- Splits routes by feature (e.g., user, product).
- Easier to manage in larger applications.
- Improves readability and reusability.

2. Basic Example

a) Create a Router File (userRoutes.js)

```
const express = require('express');
const router = express.Router();

// GET /users
router.get('/', (req, res) => {
  res.send("Get all users");
});

// POST /users
router.post('/', (req, res) => {
  res.send("Create a new user");
});

module.exports = router;
```

b) Use Router in Main File (app.js)

```
const express = require('express');
const app = express();
const userRoutes = require('./routes/userRoutes');

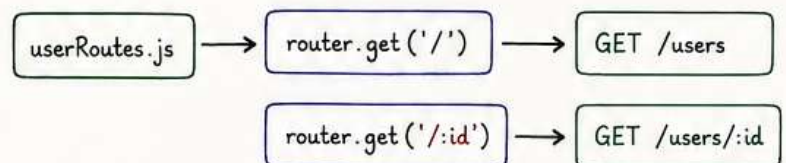
// Mount the router
app.use('/users', userRoutes);

const PORT = 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

3. Route Paths

- When we mount the router using `app.use('/users', userRoutes)`, all routes in userRoutes will start with `/users`.

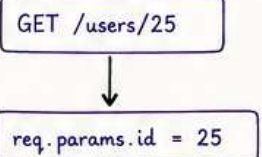


4. Route Parameters

- We can use route parameters to capture dynamic values.

```
router.get('/:id', (req, res) => {
  const userId = req.params.id;
  res.send("User ID is ${userId}");
});
```

Example :

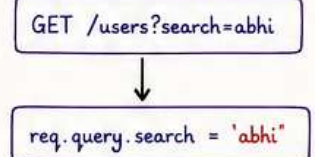


5. Query Parameters

- Access query parameters using `req.query`

```
router.get('/', (req, res) => {
  const search = req.query.search;
  res.send("Search : ${search}");
});
```

Example :



6. Multiple Routers

- We can create multiple routers for different features.



7. Best Practices

- ✓ Keep routes in separate files.
- ✓ Use meaningful route names.
- ✓ Follow RESTful conventions.
- ✓ Validate and sanitize inputs.
- ✓ Handle errors properly.



Key Takeaways



- Router helps in modular and scalable application.
- `app.use()` is used to mount routers.
- Routes can have parameters and query strings.
- Follow REST principles for clean APIs.



Tip

Always test your API using Postman or Thunder Client to ensure correct routes and responses.

EXPRESS.JS ROUTERS (Contd.)

8. NESTED ROUTERS

- We can nest routers inside other routers.

```
// userRoutes.js
const express = require('express');
const userRouter = express.Router();

userRouter.get('/', (req, res) => {
  res.send("User Home");
});

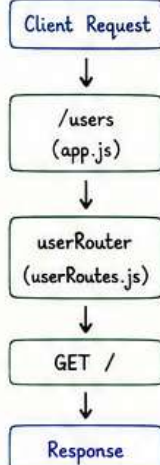
module.exports = userRouter;
```

```
// app.js
const express = require('express');
const app = express();
const userRoutes = require('./routes/userRoutes');

app.use('/users', userRoutes);

// /users will handle everything in userRoutes
app.listen(5000, () => console.log('Server running'));
```

Request Flow (Nested Router)



9. ROUTER MIDDLEWARE

- We can add middleware specific to a router.

```
const express = require('express');
const router = express.Router();

// Middleware for this router only
router.use((req, res, next) => {
  console.log("Time:", Date.now());
  next(); // pass control to next middleware/route
});

router.get('/', (req, res) => {
  res.send("Hello from router!");
});

module.exports = router;
```

Output in Console:

Time: 1715590000000 (logs for every request)

10. ROUTER PARAMS + QUERY TOGETHER

- Example of using both params and query in one route.

```
router.get('/:id', (req, res) => {
  const id = req.params.id;
  const search = req.query.search;
  res.send(`ID: ${id}, Search: ${search || 'none'}`);
});
```

Example Request

GET /users/25?search=abhi

Response

ID: 25, Search: abhi

11. ROUTER WITH MULTIPLE METHODS

- A route path can have multiple HTTP methods.

```
router.route('/:id')
  .get((req, res) => {
    res.send("Get user");
  })
  .put((req, res) => {
    res.send("Update user");
  })
  .delete((req, res) => {
    res.send("Delete user");
  });
```

Example

→ GET /users/25

→ PUT /users/25

→ DELETE /users/25

12. ORGANIZED PROJECT STRUCTURE (WITH ROUTERS)

```
my-app/
├── routes/
│   ├── userRoutes.js
│   ├── productRoutes.js
│   └── orderRoutes.js
├── middleware/
│   └── authMiddleware.js
├── app.js
└── package.json
```

Description

- routes/ → All route files
- middleware/ → Custom middleware
- app.js → Main entry file
- package.json → Project config

13. COMMON MISTAKES ❌

- Forgetting to **export** the router.
- Not mounting the router using **app.use()**.
- Wrong** route path or order of middlewares.
- Not sending a response in a route (will hang the request).
- Using **req.params** when the route doesn't have params.
- Not validating or sanitizing inputs.



Key Takeaways

- ✓ Routers help in organizing routes in a modular way.
- ✓ Use **app.use()** to mount routers.
- ✓ Routers can have their own middleware.
- ✓ Use **params** and **query** to capture dynamic data.
- ✓ Follow RESTful principles and best practices.

★ Tip

Use Postman or Thunder Client to test all routes and ensure correct responses.



EXPRESS.JS ROUTERS (Contd.)

14. ROUTER PARAMS VS QUERY

- Use params for identifiers, query for filters/search.

Route Params	Query Params
- Part of the URL path - Used to identify a resource - Example: GET /users/25	- After ? in the URL - Used for filtering, searching, pagination, etc. - Example: GET /users?search=abhi&page=1

15. ROUTER PARAMS WITH VALIDATION

- Validate and sanitize route parameters.

```
router.get('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);
  if (isNaN(id)) {
    return res.status(400).send("Invalid ID");
  }
  // Proceed with valid id
  res.send("User ID is valid");
});
```

Example

GET /users/abc
↓
400 Invalid ID

16. DEFAULT ROUTE (FALLBACK)

- Handle undefined routes in a router.

```
router.use((req, res) => {
  res.status(404).send("Route not found");
});
```

If no route matches, this will handle it.

17. ROUTER LEVEL MIDDLEWARE

- Use middleware only for a specific router.

```
const express = require('express');
const router = express.Router();

// Middleware for this router
router.use((req, res, next) => {
  console.log("Time:", Date.now());
  next();
});
```

This middleware runs for every request that comes to this router.

18. CHAINING ROUTES

- Chain middleware and route handlers.

```
router.route('/users')
  .get((req, res) => {
    res.send("Get users");
  })
  .post((req, res) => {
    res.send("Create user");
  })
  .put((req, res) => {
    res.send("Update users");
  })
  .delete((req, res) => {
    res.send("Delete users");
  });
```

All methods for /users in one place!



19. ROUTER WITH PRE-CONDITIONS

- Apply middleware before specific routes.

```
router.get('/admin/*', authMiddleware, (req, res) => {
  res.send("Admin Area");
});

function authMiddleware(req, res, next) {
  if (!req.headers.authorization) {
    return res.status(401).send("Unauthorized");
  }
  next();
}
```

authMiddleware runs before the route handler.

20. ROUTER EXAMPLE - USER MANAGEMENT

GET	/users	→ Get all users
GET	/users/:id	→ Get user by id
POST	/users	→ Create a new user
PUT	/users/:id	→ Update user
DELETE	/users/:id	→ Delete user

21. FILE STRUCTURE EXAMPLE

```
my-app/
|-- routes/
|   |-- userRoutes.js
|   |-- productRoutes.js
|-- middleware/
|   |-- authMiddleware.js
|-- app.js
|-- package.json
```

Description

- routes/ → All route files
- middleware/ → Custom middleware
- app.js → Main application file
- package.json → Project config

22. TIPS & BEST PRACTICES

- ✓ Use routers to keep routes modular and clean.
- ✓ Use meaningful route and file names.
- ✓ Use route params for identifiers, query for filters.
- ✓ Validate and sanitize all user inputs.
- ✓ Handle errors with proper status codes.
- ✓ Use middleware to keep logic DRY.
- ✓ Keep route files focused on one resource/feature.

COMMON STATUS CODES

200	OK
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Internal Server Error



Tip

Think of routers as separate mini-apps. They help in scaling your application in a clean and organized way!



EXPRESS.JS MIDDLEWARE

Middleware functions have access to the request object (**req**), the response object (**res**), and the **next** middleware function in the application's request-response cycle.

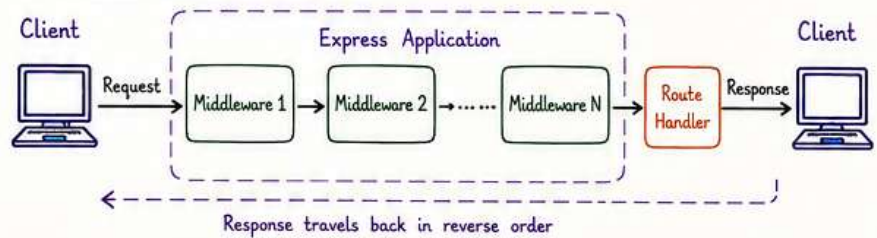
Middleware Types

- ★ Application-level
- ★ Router-level
- ★ Error-handling
- ★ Built-in
- ★ Third-party

1. What is Middleware?

- Middleware functions are functions that have access to the **req**, **res**, and **next** objects.
- They can execute any code, make changes to the request and response objects, end the request-response cycle, or call the **next** middleware function.
- They are executed in the order they are defined.

2. How Middleware Works?



3. Basic Middleware Example

```

const express = require('express');
const app = express();

// Middleware function
app.use((req, res, next) => {
  console.log("Middleware executed");
  next(); // Pass control to next middleware
});

app.get('/', (req, res) => {
  res.send("Hello from Express!");
});

app.listen(5000, () => console.log('Server running on 5000'));
  
```

Output in Console

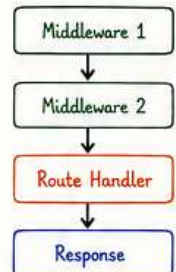
Middleware executed
↓
Hello from Express!

4. Order of Execution

- Middleware functions are executed in the order they are added using **app.use()**.
- The route handler runs after all middleware before it.
- If **next()** is not called, the request will be stuck.

Example Flow

1. Middleware 1
2. Middleware 2
3. Route Handler
4. Response sent back



5. Types of Middleware with Examples

Type	Description & Example
Application-level	Bound to an instance of app using app.use() . Example: Logging, Authentication
Router-level	Bound to an instance of express.Router() . Example: Admin routes, API routes
Error-handling	Has 4 arguments (err, req, res, next). Example: Handle errors gracefully
Built-in	Provided by Express (e.g., express.json()). Example: Parsing JSON bodies
Third-party	Installed via npm (e.g., cors , morgan). Example: CORS, Logging

6. Common Built-in Middleware

Middleware	Purpose	Usage
express.json()	Parses JSON request body	app.use(express.json())
express.urlencoded({ extended: true })	Parses URL-encoded body	app.use(express.urlencoded({ extended: true })))
express.static('public')	Serves static files	app.use(express.static('public'))
express.text()	Parses text bodies	app.use(express.text())

7. Error-Handling Middleware

```

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something went wrong!");
});
  
```

Always place error-handling middleware at the end of your middleware stack. ★

8. Third-Party Middleware Example (morgan)

```

const morgan = require('morgan');
app.use(morgan('dev'));
// Logs HTTP requests in the console
  
```

npm install morgan



Key Takeaway

- Middleware functions add powerful features to your Express apps.
- Use them for logging, authentication, validation, error handling, and more.
- Always call **next()** unless you end the response cycle.



Best Practice

Keep middleware functions small and focused on a single responsibility. ✓

EXPRESS.JS MIDDLEWARE (Contd.)

9. CUSTOM MIDDLEWARE EXAMPLE

```
const logRequest = (req, res, next) => {
  // Log method, URL and time
  console.log(`${new Date().toISOString()}
    ${req.method} ${req.url}`);
  next(); // Pass control to next middleware
};
```

★ Always call `next()` when your middleware finishes its task (unless the response is sent).

10. REGISTERING MULTIPLE MIDDLEWARE

```
const express = require('express');
const app = express();

app.use(logRequest); // Custom middleware
app.use(express.json()); // Built-in middleware
app.use(authMiddleware); // Another custom middleware

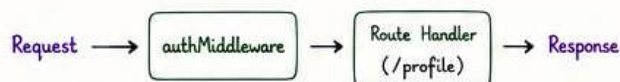
app.get('/', (req, res) => {
  res.send("Home Page");
});
```

⚠ Middleware are executed in the order they are registered.

11. MIDDLEWARE WITH ROUTE

- Apply middleware to specific routes.

```
app.get('/profile', authMiddleware, (req, res) => {
  res.send("User Profile");
});
```



★ `authMiddleware` will run only for `/profile` route.

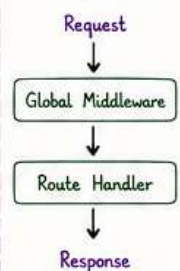
12. MIDDLEWARE WITH `app.use()` AND ROUTE

- `app.use()` without path applies to all routes.

```
app.use((req, res, next) => {
  console.log("This runs for all requests");
  next();
});

app.get('/about', (req, res) => {
  res.send("About Page");
});
```

Example Flow



13. SKIPPING TO NEXT ROUTE

- Use `next('route')` to skip remaining middleware of the current route and move to next matching route.

```
app.get('/user', (req, res, next) => {
  const id = req.query.id;
  if (!id) return next('route'); // Skip to next route
  res.send("User ID is ${id}");
});

app.get('/user', (req, res) => {
  res.send("No ID provided");
});
```



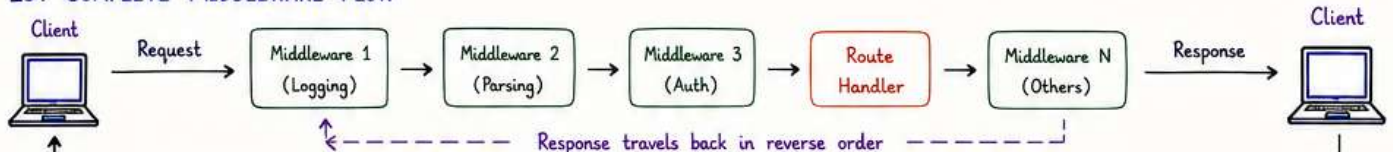
14. ERROR-HANDLING MIDDLEWARE

- 4 arguments (`err`, `req`, `res`, `next`) middleware handles errors.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something went wrong!");
});
```

💡 Always place error-handling middleware at the end of your middleware stack.

15. COMPLETE MIDDLEWARE FLOW



Best Practices

- ✓ Keep middleware small and focused.
- ✓ Use built-in middleware whenever possible.
- ✓ Use meaningful names for middleware.
- ✓ Place error-handling middleware at the end.
- ✓ Avoid calling `next()` multiple times.
- ✓ Don't forget to call `next()` if response is not sent.



Key Takeaways

- Middleware functions have access to `req`, `res` and `next`.
- They execute in the order they are registered.
- Middleware can be used globally, for routes, or for specific requests.
- Built-in middleware in Express saves a lot of time.
- Proper use of middleware makes your app modular, clean and scalable.



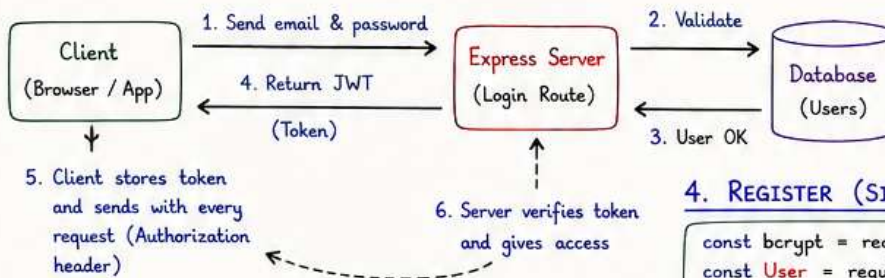
EXPRESS.JS AUTHENTICATION

Authentication is the process of verifying who a user is.
In Express apps, **JWT (JSON Web Token)** is the most commonly used method.

What is JWT ?

- JWT is a compact, URL-safe token.
- It contains claims about the user.
- It is signed using a secret key.
- It can be verified by the server.

1. TYPICAL LOGIN FLOW



2. JWT STRUCTURE

A JWT has 3 parts separated by dots (.)



Example:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6MSwiZW1haWwiOiJhaGhpQG1haWwueY29tIn0.
U1KpX9...abc123signature

3. LOGIN EXAMPLE (GENERATE JWT)

```

const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(400).send('Invalid credentials');
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(400).send('Invalid credentials');

  // Generate JWT
  const token = jwt.sign({ id: user._id, email: user.email },
    process.env.JWT_SECRET, { expiresIn: '1d' });

  // Send token
  res.json({ token });
});
  
```

4. REGISTER (SIGNUP) EXAMPLE

```

const bcrypt = require('bcrypt');
const User = require('../models/User');

app.post('/register', async (req, res) => {
  const { email, password } = req.body;
  try {
    const existing = await User.findOne({ email });
    if (existing) return res.status(400).send('Email already registered');
    // Hash password
    const salt = await bcrypt.genSalt(10);
    const hashPassword = await bcrypt.hash(password, salt);
    // Save user
    const user = new User({ email, password: hashPassword });
    await user.save();
    res.status(201).send('User registered successfully');
  } catch (err) {
    res.status(500).send('Server error');
  }
});
  
```

6. PROTECTED ROUTE (VERIFY JWT)

```

const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const header = req.headers['authorization'];
  if (!header) return res.status(401).send('No token, access denied');
  const token = header.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // attach user info to request
    next();
  } catch (err) {
    res.status(403).send('Invalid or expired token');
  }
}

// Use in routes
app.get('/profile', authenticateToken, (req, res) => {
  res.json({ message: 'Hello', user: req.user });
});
  
```



JWT + BCrypt = Secure Authentication
in Express.js Apps



BEST PRACTICES

- ✓ Always hash passwords using bcrypt.
- ✓ Never store JWT secret in code (use .env).
- ✓ Set token expiration (e.g. 1d, 7d).
- ✓ Use HTTPS in production.
- ✓ Send token in HTTP-Only cookies for more security.
- ✓ Validate and sanitize all inputs.



EXPRESS.JS AUTHENTICATION (Contd.)

7. PASSWORD HASHING WITH BCRYPT

- Never store plain text passwords.
- Use bcrypt to hash passwords before saving.
- It automatically handles salting.

```
// Install
npm install bcrypt

// Hash Password
const bcrypt = require('bcrypt');
const saltRounds = 10;
const password = 'MyStrongPassword123';
const hash = await bcrypt.hash(password, saltRounds);
console.log(hash);

// Compare Password
const isMatch = await bcrypt.compare(
  enteredPassword, hash
);
console.log(isMatch); // true or false
```

10. PROTECTED ROUTE EXAMPLE

```
const jwt = require('jsonwebtoken');
function protectRoute(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader) return res.status(401).send('No token, access denied');

  const token = authHeader.split(' ')[1]; // Bearer TOKEN
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // attach user data to request
    next();
  } catch (err) {
    res.status(403).send('Invalid or expired token');
  }
}

// Use in routes
app.get('/dashboard', protectRoute, (req, res) => {
  res.json({ message: `Welcome ${req.user.email}` });
});
```

13. TOKEN EXPIRY & REFRESH (OPTIONAL)

- Short-lived access token (e.g., 15m, 30m).
- Long-lived refresh token (e.g., 7d, 30d).
- Use refresh token to get a new access token.



★ Store refresh tokens in DB and invalidate them on logout.

8. PASSWORD CHECK DURING LOGIN

```
const user = await User.findOne({ email });
if (!user) return res.status(400).send('User not found');

const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) return res.status(400).send('Invalid password');

// If matched, generate token
const token = jwt.sign({ id: user._id, email: user.email },
  process.env.JWT_SECRET, { expiresIn: '1d' });
res.json({ token, user: { id: user._id, email: user.email } });
```

9. STORE TOKEN IN CLIENT (BEST PRACTICE)

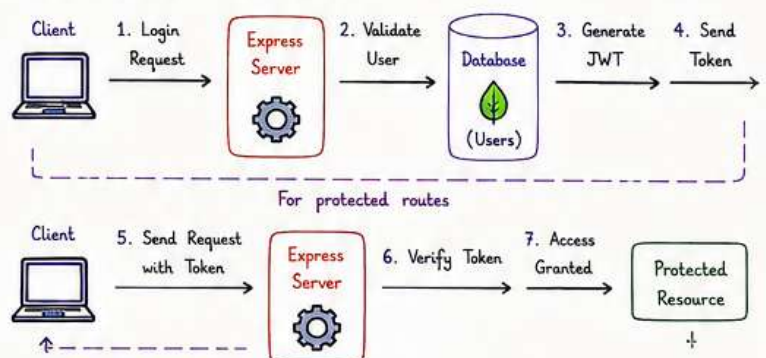
- Store JWT in **localStorage** (web) or Secure Storage (mobile).
- Do not store in cookies unless using **httpOnly** cookies.
- Attach token in Authorization header for protected routes.

Authorization: Bearer <your_token>

11. ENV VARIABLES (EXAMPLE)

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/authapp
JWT_SECRET=supersecretkey123
JWT_EXPIRES_IN=1d
```

12. COMPLETE AUTH FLOW



14. LOGOUT (SIMPLE APPROACH)

- For JWT (stateless), logout is handled on client by removing token.
- For enhanced security, you can maintain a blacklist.

```
// Add token to blacklist in DB
const Blacklist = require('./models/Blacklist');
await Blacklist.create({ token, createdAt: Date.now() });
res.send('Logged out successfully');
```

💡 On every request, check if token is blacklisted.

BEST PRACTICES



- ✓ Hash passwords using bcrypt.
- ✓ Use .env for secrets.
- ✓ Use HTTPS in production.
- ✓ Use short-lived JWT.
- ✓ Validate & sanitize all inputs.
- ✓ Store tokens securely on client.
- ✓ Implement proper error handling.
- ✓ Keep dependencies updated.



TIP

Security is not a feature, it's a mindset.

